BharatEdge-Eval: Project Write-up and Literature Review

Oct 15, 2026 · @Ro

Brutal re-evaluation

The project drops from 42/45 to 33/45. It is still worth doing, but only as a narrow, rigorous measurement study that leads into a fix, not as the broad benchmark the first report described.

The first assessment scored the idea, not the execution. Six problems were underweighted:

1. **The judge problem is the biggest threat.** No validated automatic judge exists for safety or answer quality in Hinglish and most Indic languages. Without native-speaker labels, every safety and quality number is contestable, and human labelling is the slowest, most expensive part for a solo builder.
2. **Low-end phones collapse the grid.** A 4 GB phone leaves about 1.4 GB for an app. Most 3B models and most 8-bit variants will not load. The "device × model × precision" grid on real budget phones shrinks to a handful of sub-2B models at 4-bit and below.
3. **Phone energy numbers are weak.** Android's BatteryManager readings are coarse and noisy. Without a USB power meter and many repeats, an energy claim will not survive review. Energy should be a secondary claim, not the headline.
4. **Novelty is a combination, not a discovery.** Each ingredient is published: multilingual quantization loss, safety loss under quantization, the Indic token tax. Reviewers may call the result predictable. Several 2026 preprints sit close, so the scoop risk is real.
5. **A benchmark alone signals the wrong skill.** Hiring teams want people who ship compressed models that work, not only people who measure them. Without the calibration fix, the job-market value is lower than claimed.
6. **Benchmarks are not businesses.** Startup potential was overstated. Grants and reputation are realistic; revenue is not.

Criterion	Old score	New score	Reason for the change
Novelty	4	3	Novel intersection, but each finding is individually known; scoop risk from 2026 preprints
10-year relevance	5	4	The problem lasts, but a static benchmark goes stale as models change monthly
"AI as normal" impact	5	4	Real, but only if model builders adopt the findings
Compute feasibility	5	4	Fine on the Mac; the phone grid and human labelling are the real costs
Solo feasibility	4	3	Human evaluation in several languages is hard alone
Resume fit	5	5	Value Imprint, edge ML and evals still fit exactly
Job-market signal	5	4	Strong only if the fix (calibration) ships alongside the measurement
Startup or grant potential	4	2	Grant-fundable; weak commercial path
Time to first demo	5	4	Pilot in a week is realistic; the full study is 8–12 weeks, not 4–8
Total (out of 45)	**42**	**33**	

Verdict: proceed, with the scope cuts in the Scope section and these gates.

- [ ] Gate 1 (end of week 1): the pilot shows a gap amplification factor above 1.5 for at least one language at 4-bit on at least two models. If not, the core claim fails and we stop.
- [ ] Gate 2 (end of week 2): an LLM judge agrees with a native speaker on at least 80% of 200 Hindi and Hinglish items. If not, we restrict claims to multiple-choice and reference-based metrics.
- [ ] Gate 3 (end of week 4): multilingual calibration recovers a measurable share of the loss. If not, we publish the measurement alone as a negative-result-friendly study.

How to raise the score

The project can credibly reach about 41/45, not 45; each point below is earned only when its evidence exists, not by adding it to the plan.

Two points stay out of reach on purpose. A 5 for novelty needs a finding nobody predicted, which no plan can guarantee. A 5 for startup potential needs paying customers, which a research study cannot produce in 12 weeks.

Criterion	Now	Target	What earns the point	Evidence that proves it
Novelty	3	4	Make the calibration fix a named method: language- and safety-aware importance-matrix calibration, tested on Hinglish, which no prior work covers	H4 passes on at least 2 of 3 model families
10-year relevance	4	5	Turn the benchmark into a living harness: a weekly job that tests new community GGUF releases and posts the language gap automatically	The harness runs unattended for 4 weeks and catches at least one new model
"AI as normal" impact	4	5	Publish recalibrated GGUF models on Hugging Face and open a pull request adding multilingual calibration text to llama.cpp	Downloads of the released models, or the pull request reviewed or merged
Compute feasibility	4	5	Time the smoke grid on day 1 and budget the full grid from real numbers	The full grid finishes inside the measured budget
Solo feasibility	3	4	Recruit 2–3 paid native-speaker labellers and one academic co-author; cut to English, Hindi and Hinglish if labelling falls behind	200 labelled items with agreement statistics by week 2

Criterion	Now	Target	What earns the point	Evidence that proves it
Resume fit	5	5	Already maxed	—
Job-market signal	4	5	Ship a small Android demo that runs the same Hindi question on the default and the recalibrated model side by side	A 60-second screen recording and the public app build
Startup or grant potential	2	3	Apply to Emergent Ventures India and pitch the harness to Bhashini or AI4Bharat as a public evaluation asset; offer "language-parity checks" to one Indian team shipping on-device AI	One grant decision or one team using the harness
Time to first demo	4	5	Post the pilot chart publicly within 7 days	The public post with the Gate 1 chart
Total (out of 45)	**33**	**41**		

Making the study hard to attack

- **Pre-register.** Commit the hypotheses, thresholds, item sets and analysis code to a tagged GitHub release before the full grid runs. Reviewers cannot accuse you of choosing the metric after seeing results.
- **Sanity baselines.** Report random-guess accuracy, reproduce one published BF16 English score, and confirm that 8-bit loses almost nothing. If 8-bit shows a large loss, the harness is broken, not the model.
- **Blind the labellers.** Native speakers never see which precision produced an answer.
- **Hold one language out.** Choose Bengali or Tamil, but do not look at its results until the analysis code is frozen. If the pattern holds there, it is not a fluke of Hindi.
- **Two calibration controls.** Compare multilingual calibration against English-only calibration and against no calibration, with calibration text of equal length, so any gain comes from the language mix, not from more data.
- **Outside review before posting.** Send the draft to two researchers (an AI4Bharat author and a quantization author) and fix what they find before anything goes public.

Abstract and problem statement

BharatEdge-Eval measures how much quality and safety quantized small language models lose in Hindi, romanized Hinglish and other Indic languages compared with English, and then tests whether multilingual calibration recovers that loss.

Compression is how most of the world will meet AI. A model must shrink to 4 bits or below to fit a budget phone. Quantization is usually calibrated on English text and judged by English perplexity on server hardware. Published work shows that this process hurts non-Latin scripts most, that automatic metrics understate the harm, and that safety can degrade while perplexity stays flat.

No published study, to our knowledge, measures all three together: quantization level, Indic and code-mixed input, and safety, on models small enough for low-end devices. This project fills that gap with a reproducible harness, a measured result, and a cheap fix.

Research question: at the precisions needed for budget devices, how much more do Indic and Hinglish users lose than English users in capability and safety, and how much of that loss can calibration data recover at zero inference cost?

Literature review: quantization and multilingual degradation

The literature agrees that quantization is not language-neutral: non-Latin scripts and harder tasks lose the most, and automatic metrics hide much of it.

Work	Venue	Main finding	What it means for us
Marchisio et al., How Does Quantization Affect Multilingual LLMs?	EMNLP Findings 2024	Non-Latin scripts degrade most; a 1.7% automatic drop in Japanese matched a 16.0% drop in human ratings	Automatic scores alone will understate Indic losses; human checks are required
Mekala et al.	EMNLP 2025	8-bit is near lossless; 4-bit loses up to 59% on long-context tasks, worse outside English	Expect a cliff between 8-bit and 4-bit, so sample precisions densely there

Work	Venue	Main finding	What it means for us
Quantization and MT across 55 languages	arXiv 2025	Low-resource and typologically distant languages degrade most, especially at 2-bit	Include at least one lower-resource Indic language
Calibrating Beyond English	arXiv 2026	Standard calibration is English-only; multilingual calibration improves quantized models	Closest prior work to our fix; we must extend it to Hinglish and safety data
The Multilingual Quantization Tax	arXiv 2026, single author	Translated test items show far larger losses than native items	Use native-language items where possible; report translated items separately
Bangla quantization study	arXiv 2026	Effects vary sharply by model family and method	Never generalise from one model family
Saying, Not Knowing	arXiv 2026	Heavily quantized small models keep using rare words they can no longer define; community GGUF builds differ	Test the community builds people actually download, not only our own

Two gaps remain open. First, none of these studies covers romanized code-mixed text such as Hinglish, which is how many Indian users type. Second, most use models of 7B and above, while budget phones need models under 2B, where losses start earlier.

Literature review: safety, bias and alignment under compression

Two separate bodies of work show that quantization weakens safety and that safety is weakest in low-resource languages; nobody has measured the two effects together.

Safety under quantization. Kharinaev et al. tested 66 quantized variants and found that both post-training quantization and quantization-aware training degrade safety, with no method best everywhere. Alignment-Aware Quantization reports safety scores falling by more than 20 points at 4-bit weights and activations while perplexity barely moves.

QuantiBias finds refusal rates stay flat while open-ended bias rises, and Quantization Undoes Alignment reports new stereotyped answers on 2.5–5.6% of BBQ items at 4-bit. The common lesson: refusal-rate checks miss the damage, so open-ended generation must be judged.

Safety in low-resource languages. MultiJail (ICLR 2024) found low-resource languages roughly three times as likely to produce harmful content. IndicSafeEval built 7,200 persuasive jailbreak prompts in Hindi, Bengali, Marathi and Punjabi. IndicGuard offers a 4B Indic safety classifier, a candidate judge.

The comprehension trap. A study of jailbreaks on small models warns that a small model can look safe in a weak language simply because it did not understand the request. Any safety score we report must separate "refused" from "did not understand". This needs a comprehension probe on every safety item.

Link to Value Imprint. None of these works asks whether the distribution of values in a model's answers shifts with compression. Our NeurIPS 2024 classifier can measure that directly, which is the project's one fully original measurement.

Literature review: tokenization tax for Indic scripts

Indic text costs several times more tokens than English for the same meaning, and on a phone every extra token is extra time and battery.

The Tokenizer Tax measures a mean 8.0× token inflation for Indian languages under a common English-centric tokenizer, reaching 13.0× for Malayalam, which cuts the usable context window to as little as 12% of English. Multilingual tokenizers remove about 73% of that tax. Removable and Irreducible argues most of the excess is a design choice, not a property of the languages. Older work (Petrov et al. 2023; Ahia et al. 2023) established the same pattern across many languages.

The small models we test use different tokenizers (Gemma, Qwen and Llama each have their own), so the tax varies by model. This is a confound: a model may look slow in Hindi because of its tokenizer, not its quantization. We will report tokens per answer for every model and language, and compare cost per correct answer rather than per token.

Romanized Hinglish behaves differently again. Written in Latin script, it often tokenizes cheaply but is underrepresented in training data. Whether it suffers more or less from quantization than Devanagari Hindi is an open question this project can answer.

Literature review: on-device inference and existing benchmarks

On-device benchmarks exist, but they test English, on flagship phones, and leave safety out.

Work	What it covers	What it misses
MobileAIBench	Quantized LLMs and multimodal models on mobile, task quality and resource use	Indic and code-mixed languages; safety
SLMQuant	Quantization methods applied to small language models	Devices, languages other than English, safety
Demystifying SLMs for Edge (ACL 2025)	Small models on real phones, including latency spikes on an older device under sustained load	Multilingual quality and safety
Sustainability Is Not Linear	Energy and privacy trade-offs on device, non-intrusive battery measurement	Low-end phones (uses flagships)
BitNet b1.58 2B4T	A native ternary 2B model with very low memory and estimated energy	Measured phone energy; Indic quality

The on-device runtimes that matter are llama.cpp with GGUF files (the default for open models), MLX on Apple silicon, and ExecuTorch, MLC-LLM and LiteRT on mobile. Google's Gemma 3n is designed to run in about 2–3 GB of memory, and Sarvam's Edge stack targets Indic languages offline on feature phones.

One practical finding shapes our device plan: a developer test on a 4 GB Android phone found only about 1.4 GB available to an app, with the system killing inference mid-run. Datasheet RAM is not usable RAM.

Synthesis: the gap and revised hypotheses

The gap is narrow and real: no study measures quantization loss in Indic and code-mixed languages for capability, safety and values together on sub-2B models, and then tests a calibration fix.

The hypotheses are cut to four, each with a number that decides it.

1. **H1, gap amplification.** At 4-bit, the relative quality loss in Hindi and Hinglish is more than 1.5 times the English loss, for at least two of three model families. Measured by the gap amplification factor (defined in the protocol).
2. **H2, hidden safety loss.** At 4-bit and below, refusal of direct harmful prompts stays within 5 points of the full-precision model, while judged harm in open-ended Hindi or Hinglish answers rises by more than 5 points. Items the model did not understand are excluded.
3. **H3, value drift.** The distribution of Value Imprint categories in answers shifts more from full precision in Hindi than in English (KL divergence, with bootstrap confidence intervals).
4. **H4, calibration fix.** Adding Hindi, Hinglish and safety text to the calibration data recovers at least a third of the H1 gap at 4-bit, at zero extra inference cost.

Dropped from the first plan: the headline energy-per-answer claim on phones (too noisy without a power meter) and the tokenizer front-end ablation (a separate project). Phone measurements stay, but only as a secondary feasibility result: which configurations load and how fast they run.

Scope: what we will and will not do

The study is deliberately small: 3 model families, 5 precisions, 4 languages, 3 task types, measured on a Mac with a short phone feasibility run.

In scope

- Models under 2B parameters: Gemma 3 1B, Qwen3 1.7B, Llama 3.2 1B. Qwen3 0.6B as a size check.
- Precisions: BF16 reference, Q8_0, Q6_K, Q4_K_M, Q3_K_M, plus the most-downloaded community Q4 build of each model.
- Languages: English (control), Hindi in Devanagari, romanized Hinglish, and one more Indic language (Bengali or Tamil, chosen by data availability).
- Tasks: multiple-choice reading comprehension (scored by log-likelihood, no judge needed), short generative answers (judged), and safety prompts (judged, with a comprehension probe).
- Value drift via the Value Imprint classifier.
- The calibration fix (H4) using llama.cpp's importance-matrix calibration.

- A phone feasibility table: which configurations load, time to first token, tokens per second, peak memory, on one 4 GB Android phone.

Out of scope (and why)

Not doing	Reason
Models of 3B and above	They do not fit the target phones; Mac time is better spent on more seeds
Measured energy as a headline claim	Phone battery readings are too noisy without a USB power meter
Training or fine-tuning models	Calibration needs no training; fine-tuning adds cost and confounds the comparison
New tokenizers or transliteration front ends	A separate project; would blur what causes any gain
Voice or speech	Sarvam and Google already cover offline Indic speech; outside the research question
A public crowdsourced phone app	Phase 3 at the earliest; needs privacy review
Releasing jailbreak outputs	Safety risk; only aggregate scores and gated prompts are released
GPTQ, AWQ and quantization-aware training	GGUF is what people actually run on phones; one method family keeps the study clean

Technology: what we will and will not use

One runtime does all quantization and inference: llama.cpp with GGUF files, on the Mac's Metal GPU, so every number comes from the same code path users actually run.

Stack we will use

Layer	Tool	Why
Quantization	llama.cpp `llama-quantize` and `llama-imatrix`	GGUF is the format phones run; importance-matrix calibration is the H4 lever

Layer	Tool	Why
Inference and scoring	llama-cpp-python (Metal build)	Gives token log-probabilities for multiple-choice scoring and plain generation for judged tasks
Model download	`huggingface_hub`	Official weights plus community GGUF builds
Environment	Python 3.11, `uv`, exact versions pinned in a lock file	Reproducible runs; llama.cpp built from a recorded commit hash
Config and runs	YAML configs, one JSONL file of results per run, DuckDB for analysis	No server, no account, easy to re-run any cell of the grid
Judges	IndicGuard for safety; one local instruction model (about 8–12B, 4-bit) for answer quality; a free-tier API model as a second opinion	Two independent judges plus human labels; no single judge is trusted
Value drift	The Value Imprint classifier, PyTorch on Apple MPS	Reuses the NeurIPS 2024 work directly
Statistics and plots	NumPy, SciPy, pandas, Matplotlib	Bootstrap intervals, paired permutation tests, Holm correction
Phone runs	Termux with llama.cpp built on the device, `adb` for logs	Same runtime as the Mac, so phone and Mac numbers are comparable
Quality gates	`pytest`, `ruff`, a small fixed smoke-test grid	Catch broken runs before spending hours on the full grid
Overflow compute	Kaggle (about 30 GPU hours per week) for large judge batches only	The Mac handles all sub-2B inference

Tools we will not use

Tool	Reason
MLX quantization	A second quantization format would confound the comparison; GGUF only
PyTorch MPS for model inference	Operator fallbacks and precision quirks; used only for the small classifier
vLLM, TensorRT-LLM, bitsandbytes, AWQ, GPTQ	Built for NVIDIA GPUs, not phones; out of scope
Docker for inference on the Mac	No Metal GPU access inside containers
Android emulators for speed or memory numbers	Cannot reproduce real memory limits or heat
Weights and Biases, MLflow	Account and setup overhead for a solo study; JSONL plus DuckDB is enough
LangChain, LangGraph	No agent or chain logic is needed
Paid frontier models as the main judge	Cost, and results others cannot reproduce; allowed only as a spot check
Fine-tuning libraries (PEFT, TRL)	No training in this study

Experimental protocol

Every model and precision is run on the same fixed item sets, in the same order, with greedy decoding, so any difference comes from quantization alone.

Grid. 4 models × 6 builds (BF16, Q8_0, Q6_K, Q4_K_M, Q3_K_M, community Q4) = 24 configurations, plus 8 recalibrated Q4_K_M and Q3_K_M builds for H4. Total 32.

Data (licences and exact splits to be confirmed in week 1):

Task	Source	Languages	Items per language	Scoring
Reading comprehension	Belebele	English, Hindi (Devanagari), romanized Hindi, Bengali or Tamil	900	Log-likelihood over 4 options; no judge

Task	Source	Languages	Items per language	Scoring
Short generation	IndicGenBench subsets; a Hinglish set from Indi-RomCoM or CodeMixBench	All four	200	Two judges plus reference metrics
Harmful requests	IndicSafeEval (Hindi), MultiJail (Bengali), translated and native-checked for Hinglish	All four	300	IndicGuard plus second judge
Over-refusal	XSTest, translated and native-checked	All four	150	Judge: refused or answered
Comprehension probe	One paraphrase question per safety item	All four	Matches safety set	Correct or not

Primary metric. Relative loss for language L at precision q:

$$\Delta_{L,q} = \frac{S_{L,\mathrm{BF16}} - S_{L,q}}{S_{L,\mathrm{BF16}}}$$

The gap amplification factor is the ratio of the Indic loss to the English loss. A ratio is unstable when the English loss is near zero, so the primary test uses the difference, and the ratio is reported only when the English loss exceeds 1 point:

$$G_{L,q} = \Delta_{L,q} - \Delta_{\mathrm{en},q}$$

Safety metrics. Harmful-compliance rate on items the model understood, over-refusal rate, and the share of items not understood, all reported separately.

Value drift. KL divergence between Value Imprint category distributions at BF16 and at each precision, per language.

Statistics. 1,000-sample bootstrap 95% confidence intervals; paired permutation tests between precisions on identical items; Holm correction across languages; 3 sampling seeds on a 50-item subset to check that greedy results are stable.

Human check. A native speaker labels 200 Hindi and Hinglish items (quality and safety). Judge-human agreement must reach 80% (Gate 2) before judged numbers are reported as findings.

Phone run. One 4 GB Android phone, airplane mode, battery above 50%, 3 warm-up runs, 5 timed runs per configuration: load success, time to first token, decode speed, peak memory, and whether the system killed the process.

Claude Code execution plan

Give the Claude Code session this document plus the rules below, and have it build milestone by milestone, stopping at each acceptance check before moving on.

Repository layout

```
bharatedge-eval/
  CLAUDE.md              # rules for the agent (below)
  pyproject.toml, uv.lock
  configs/               # models.yaml, precisions.yaml, tasks.yaml,
grid.yaml
  data/                  # download scripts only; raw data is never committed
  src/bee/
    quantize.py          # GGUF conversion, quantization, imatrix calibration
    runner.py            # loads a config, runs one task, writes JSONL
    tasks/               # belebele.py, generation.py, safety.py,
overrefusal.py, probe.py
    judges/              # indicguard.py, local_judge.py, api_judge.py,
agreement.py
    values.py            # Value Imprint classifier wrapper
    stats.py             # bootstrap, permutation tests, Holm correction
    report.py            # tables and plots from DuckDB
  phone/                 # Termux build notes and a timing script
  results/               # JSONL per run, gitignored except summaries
  tests/                 # smoke grid and unit tests
```

Rules for CLAUDE.md

- Never change a task's item set or prompt template after Milestone 2 without a new version number.
- Every run records the llama.cpp commit, GGUF file hash, config hash and seed in its JSONL header.
- Run the smoke grid (1 model, 2 precisions, 20 items per task) before any full run.

- Never print or commit harmful model outputs; store them only in the gitignored results folder.
- If a result looks surprising, re-run that cell before writing anything about it.

Milestones and acceptance checks

1. **Setup.** Build llama.cpp with Metal, create the environment, download the 4 models. *Done when* each BF16 model answers one English and one Hindi prompt from the command line.
2. **Quantize and freeze data.** Produce the 24 GGUF builds; download and version all task data. *Done when* file hashes are recorded and the smoke grid passes.
3. **Multiple-choice harness.** Belebele log-likelihood scoring. *Done when* BF16 English accuracy for each model is within 3 points of any published figure, or the gap is explained.
4. **Pilot and Gate 1.** Full Belebele grid. *Done when* the loss chart exists and Gate 1 is decided.
5. **Judges and Gate 2.** Generation, safety, over-refusal and probe tasks; two judges; native-speaker labelling of 200 items. *Done when* agreement is reported and Gate 2 is decided.
6. **Full grid and value drift.** All 24 configurations on all tasks, plus Value Imprint scoring. *Done when* every cell has a result with a confidence interval.
7. **Calibration fix and Gate 3.** Build multilingual and safety imatrix files; re-quantize 8 builds; re-run. *Done when* H4 is decided.
8. **Phone feasibility.** Termux runs on the 4 GB phone. *Done when* the feasibility table is complete.
9. **Write-up and release.** Code, configs, aggregate results, a short paper and a blog post. *Done when* someone else can reproduce one table from the README alone.

Risks, kill criteria and ethics

The project stops early if the gap does not exist (Gate 1); every other risk narrows the claims but does not end the study.

Risk	Likelihood	Effect	Response
No meaningful gap at 4-bit (Gate 1 fails)	Medium	Core claim fails	Stop; publish a short negative note; switch to the language-fair compression toolkit
Judges disagree with native speakers (Gate 2 fails)	High	Safety and generation numbers become contestable	Report only multiple-choice and reference metrics as findings; judged results become exploratory
A close paper appears first	Medium	Novelty shrinks	Release the pilot publicly within 2 weeks; differentiate on Hinglish, sub-2B models and value drift
Datasets unavailable or licence-restricted	Medium	Gaps in the language grid	Swap Bengali and Tamil; build a small native-checked set if needed
Phone too limited to load most builds	High	Thin phone table	Report it as a finding: what actually fits on a 4 GB phone
Community GGUF builds change or vanish	Low	Reproducibility	Record file hashes; mirror the exact files privately

Ethics. Harmful prompts are released only through gated access, and model outputs to them are never released. Findings are reported by model family and precision, not as a ranking of which model is easiest to misuse in Hindi. Native-speaker labellers are paid at a fair rate and see harmful content only with warning and consent.

References

Sources come from the earlier research reports in this conversation; many 2026 items are unreviewed arXiv preprints, so check each before citing it in a paper.

1. Marchisio et al., How Does Quantization Affect Multilingual LLMs?, EMNLP Findings 2024
2. Mekala et al., quantization across languages and long context, EMNLP 2025
3. Quantization and machine translation across 55 languages, arXiv 2025
4. Calibrating Beyond English, arXiv 2026
5. The Multilingual Quantization Tax, arXiv 2026
6. Quantization effects on Bangla language understanding, arXiv 2026
7. Saying, Not Knowing, arXiv 2026
8. Kharinaev et al., quantization and safety, arXiv 2025
9. Alignment-Aware Quantization, arXiv 2025
10. QuantiBias, arXiv 2026
11. Quantization Undoes Alignment, arXiv 2026
12. MultiJail, Deng et al., ICLR 2024
13. IndicSafeEval, arXiv 2026
14. IndicGuard, arXiv 2026
15. Jailbreak resistance of small language models, arXiv 2025
16. The Tokenizer Tax for Indian languages, arXiv 2026
17. Removable and Irreducible, arXiv 2026
18. Petrov et al., tokenizer unfairness between languages, NeurIPS 2023
19. Ahia et al., Do All Languages Cost the Same?, EMNLP 2023
20. MobileAIBench
21. SLMQuant, arXiv 2025
22. Demystifying Small Language Models for Edge Deployment, ACL 2025
23. Sustainability Is Not Linear, arXiv 2026
24. BitNet b1.58 2B4T, arXiv 2025
25. llama.cpp